

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа 1

Выполнил:

Берулава Леон Алхасович

К3439

Проверил:

Добряков Д. И.

Санкт-Петербург

2025 г.

Задача

Нужно написать свой boilerplate на express + TypeORM + typescript.

Должно быть явное разделение на:

- модели
- контроллеры
- роуты

Ход работы

В рамках работы создан универсальный boilerplate для разработки веб-приложений на основе Express.js с использованием TypeORM и TypeScript. Архитектура построена по принципу четкого разделения ответственности с выделением отдельных слоев для моделей данных, бизнес-логики и маршрутизации запросов.

Основная структура проекта организована следующим образом: директория src содержит весь исходный код приложения с подразделением на папки entities для моделей данных, controllers для контроллеров, routes для маршрутов, services для бизнес-логики и config для конфигурационных файлов. Такое разделение обеспечивает высокую читаемость кода, упрощает тестирование и позволяет легко масштабировать приложение при добавлении новых функций.

Конфигурация TypeScript настроена для работы с декораторами TypeORM что позволяет использовать аннотации для определения сущностей базы данных. Включена строгая типизация и экспериментальная поддержка декораторов через параметры experimentalDecorators и emitDecoratorMetadata в файле tsconfig.json. Результат компиляции помещается в директорию dist для развертывания в production среде.

Слой моделей реализован через TypeORM Entity классы расположенные в директории entities. Каждая модель представляет собой TypeScript класс с декораторами определяющими структуру соответствующей таблицы в базе данных. Используются декораторы Entity для определения таблицы, PrimaryGeneratedColumn для автоинкрементных первичных ключей, Column для обычных полей с указанием типов данных и ограничений.

Модель User демонстрирует использование различных типов полей включая уникальные ограничения для username и email, nullable поля для необязательных данных и временные метки через CreateDateColumn и UpdateDateColumn. Применены индексы через декратор Index для оптимизации поиска по часто используемым полям. Поля пароля реализованы как password_hash для безопасного хранения хэшированных значений.

Модель Recipe показывает работу с JSON типами данных для хранения массивов ингредиентов и инструкций что обеспечивает гибкость структуры данных без необходимости создания дополнительных связанных таблиц. Внешние ключи реализованы через обычные integer поля с последующим связыванием на уровне бизнес-логики для сохранения независимости микросервисов.

Контроллеры содержат бизнес-логику обработки HTTP запросов и расположены в директории controllers. Каждый контроллер представляет собой TypeScript класс с методами соответствующими различным HTTP операциям. Контроллеры получают объекты Request и Response от Express и возвращают стандартизированные JSON ответы с полями success, data, message и error.

UserController демонстрирует полный набор CRUD операций включая валидацию входных данных, обработку ошибок и взаимодействие с сервисным слоем. Методы контроллера реализованы как arrow functions для правильного связывания контекста this что позволяет использовать их напрямую в маршрутах без дополнительного биндинга. Валидация email адресов реализована через приватный метод isValidEmail с использованием регулярных выражений.

RecipeController показывает работу с более сложными сценариями включая пагинацию, фильтрацию и поиск. Параметры запроса извлекаются из req.query и req.params с последующей валидацией и приведением типов. Обработка ошибок унифицирована через try-catch блоки с возвратом стандартизированных сообщений об ошибках включая коды состояния HTTP и временные метки.

Маршруты определены в отдельных файлах в директории routes с использованием Express Router для модульной организации эндпоинтов. Каждый файл маршрутов соответствует определенной предметной области и импортирует соответствующий контроллер для обработки запросов. Маршруты определяются декларативно через методы router.get, router.post, router.put и router.delete с указанием пути и обработчика.

Файл userRoutes демонстрирует правильное упорядочивание маршрутов где более специфичные пути размещены перед параметрическими для предотвращения конфликтов маршрутизации. Маршрут /stats размещен перед /:id чтобы избежать интерпретации stats как параметра id. Аналогично маршрут /email/:email предшествует /:id для корректной обработки поиска по email адресу.

Файл recipeRoutes показывает организацию специальных маршрутов таких как /popular и /user/:userId которые предоставляют дополнительную

функциональность помимо стандартных CRUD операций. Все маршруты используют соответствующие HTTP методы согласно REST конвенциям где GET используется для получения данных, POST для создания, PUT для обновления и DELETE для удаления.

Вывод

Созданный boilerplate обеспечивает четкое разделение ответственности между слоями приложения упрощает добавление новых функций и поддерживает принципы SOLID для создания масштабируемых веб-приложений на TypeScript.